AMITY INSTITUTE OF INFORMATION TECHNOLOGY (AIIT)

Graph Data Analytics and Representation Learning (CSE5083)

Winter Semester – 2025-2026

Multi-Level Fake Account Detection in Social Networks using Graph Centrality and GNNs

Project Report

SUBMITTED TO:

Dr. Geetha V

(Assistance Professor-III)

(AIIT)

SUBMITTED BY:

Krishnam Mavani

(MSc Data Science)

(A869117725001)

Amity University Bengaluru

NH-207, opposite to the office of Deputy Commissioner, Devanahalli,

Doddaballapura, Bangalore – 562110

Multi-Level Fake Account Detection in Social Networks using Graph Centrality and GNNs

1. Introduction

Social networking platforms such as Facebook, Instagram, Twitter, and LinkedIn have become essential communication tools. However, the rapid growth of these platforms has also increased the number of fake and malicious accounts. These accounts are often used for spamming, spreading misinformation, phishing attacks, identity theft, and manipulating public opinion.

Traditional machine learning approaches mainly focus on user profile attributes and behavioural features. However, social networks naturally form graph structures where users are represented as nodes and friendships or interactions are represented as edges. Graph-based analysis provides deeper insights into user relationships and network behaviour.

This project proposes a graph-based fake account detection system that combines:

- Graph Mining Techniques
- Centrality Measures
- Community Detection
- Node Embeddings
- Graph Neural Networks (GNNs)

The objective is to identify suspicious accounts by analysing both structural properties and learned graph representations.

2. Project Objectives

The main objectives of this project are:

➝ **Primary Objectives**

- Detect fake or suspicious users in social networks.
- Analyse graph structure using graph mining techniques.
- Generate node embeddings to represent user relationships.
- Train a Graph Neural Network for classification.
- Evaluate model performance using standard metrics.

➝ **Expected Outcomes**

- Identification of fake accounts.
- Visualization of suspicious network regions.

- Performance analysis of the GNN model.
- Understanding of social network behaviour.

3. Recommended Requirements Analysis

The project requirements are fully satisfied as follows:

Requirement	Implementation
Dataset Size > 1000 Nodes	Facebook Dataset contains 4039 nodes
Graph Mining Technique	Centrality Analysis & Community Detection
Embedding Method	Node2Vec Embeddings
Graph Neural Network	GCN (Graph Convolutional Network)
Evaluation Metric	Accuracy, Precision, Recall, F1 Score
Visualization	Network Graph Visualizations
Python Implementation	Implemented using Python
Result Analysis	Included in notebook

4. Dataset Description

➞ **Dataset Name**

Dataset: Facebook Combined Social Network Dataset
Source: Stanford Network Analysis Project (SNAP)
Dataset File: Facebook_combined.txt

➞ **Dataset Statistics**

Property	Value
Nodes (Users)	4039
Edges (Connections)	88234
Graph Type	Undirected
Network Type	Social Network

The dataset satisfies the minimum requirement of 1000+ nodes.

5. Tools & Technologies Used

The project is implemented using Python and several graph analysis libraries.

➞ **Tools (Python Libraries):**

i. Pandas

Used for:

- Data loading
- Data manipulation
- Tabular processing

ii. **NumPy**

Used for:

- Numerical operations
- Matrix computations

iii. **NetworkX**

Used for:

- Graph creation
- Centrality calculation
- Community analysis

iv. **Matplotlib**

Used for:

- Graph visualization
- Result plotting

v. **Node2Vec**

Used for:

- Graph embedding generation

vi. **PyTorch**

Used for:

- Deep learning operations

vii. **PyTorch Geometric**

Used for:

- Graph Neural Network implementation

➝ **Technologies:**

i. **GNN**

Used for:

- Learning patterns from graph data
- Node classification
- Social network analysis
- Fake account detection

6. Graph Construction

➝ **Purpose**

The first step is converting the Facebook edge list into a graph structure.

Example code:

```
G = nx.read_edgelist('facebook_combined.txt')
```

➝ **Reason**

The dataset initially contains only user connections.

Graph construction allows:

- Node representation
- Edge representation
- Centrality computation
- Community detection
- GNN processing

Without converting the dataset into a graph, network analysis cannot be performed.

7. Graph Mining Techniques

Graph mining helps extract meaningful information from network structures.

i. Degree Centrality

➝ **Purpose**

Measures how many direct connections a user has.

Formula:

$$Degree(v) = \frac{deg(v)}{N-1}$$

➝ **Reason**

Fake accounts often have:

- Very few connections
- Extremely high connections due to spam behaviour

Both cases can be identified through degree centrality.

ii. Betweenness Centrality

➝ **Purpose**

Measures how frequently a node appears on shortest paths.

Formula:

$$BC(v) = \sum \frac{\sigma(s,t \mid v)}{\sigma(s,t)}$$

➝ **Reason**

Suspicious accounts sometimes act as bridges between unrelated communities.
High betweenness values indicate such behaviour.

iii. Closeness Centrality

➝ **Purpose**

Measures how quickly a user can reach other users.

➝ **Reason**

Fake accounts generally occupy peripheral positions within networks.
Low closeness values may indicate suspicious behaviour.

iv. Eigenvector Centrality

➝ **Purpose**

Measures node importance based on connections to other important nodes.

➝ **Reason**

Real users tend to connect with influential users.

Fake accounts often have low influence scores.

8. Community Detection

➝ **Purpose**

Community detection identifies groups of users that interact frequently.

Example:

```
communities = nx.community.greedy_modularity_communities(G)
```

➝ **Reason**

Fake accounts often:

- Exist outside communities
- Connect abnormally across multiple communities

Community analysis helps discover such anomalies.

9. Node Embedding Generation

➝ **Node Embedding**

Machine learning models cannot directly understand graph structures.

Node embeddings convert graph nodes into numerical vectors.

Example:

```
node2vec = Node2Vec(G)
```

➝ **Reason**

Node2Vec captures:

- Local neighbourhood information
- Global network structure
- Similar user behaviour

Generated embeddings serve as input features for the GNN.

10. Graph Neural Network (GNN)

➝ **Purpose**

Traditional neural networks ignore graph relationships.

Graph Neural Networks learn from:

- Node features
- Neighbor information
- Network structure

This makes them highly effective for social network analysis.

➝ **Graph Convolutional Network (GCN)**

The project uses a GCN model.

- **Working Principle**

For each node:

1. Gather neighbouring information
2. Aggregate features
3. Update node representation
4. Predict fake/real account

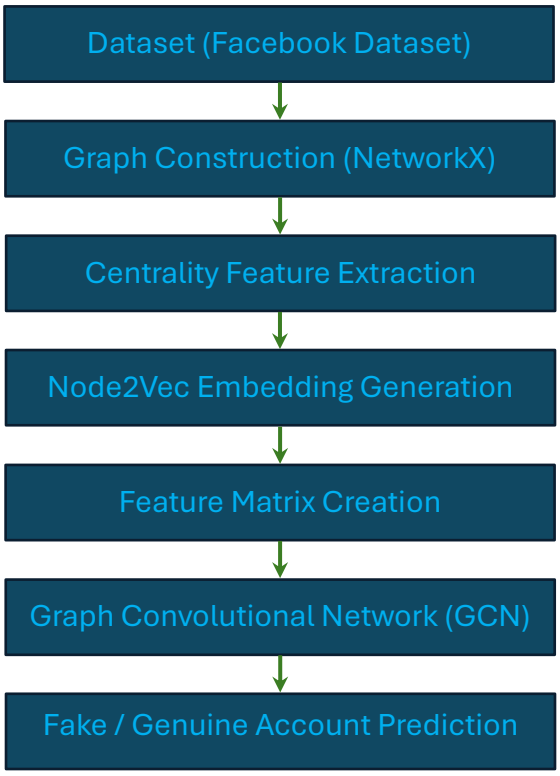
- **Why GCN is Used in This Project**
  - Effectively analyses social network data.
  - Utilizes user connections and graph structure.
  - Learns hidden relationship patterns between users.
  - Improves fake account detection accuracy.
  - Suitable for node classification tasks in social networks.

11. Model Training

During training:

1. Node embeddings are generated.
2. Centrality features are computed.
3. Features are combined.
4. Data is split into train and test sets.
5. GCN is trained.

Example workflow:



12. Evaluation Metrics

The model performance is evaluated using:

i. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Measures overall prediction correctness.

ii. Precision

$$Precision = \frac{TP}{TP + FP}$$

Measures correctness of detected fake accounts.

iii. Recall

$$Recall = \frac{TP}{TP + FN}$$

Measures how many actual fake accounts were found.

iv. F1 Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Balances precision and recall.

13. Visualization

Visualization is a mandatory project requirement.

The notebook visualizes:

i. Network Graph

Shows:

- Nodes (users)
- Edges (connections)

ii. Community Structure

Shows:

- User clusters
- Suspicious communities

iii. Centrality Distribution

Shows:

- Highly influential users
- Isolated users

iv. GNN Predictions

Shows:

- Real accounts
- Fake accounts

Visualization improves interpretability and helps verify model behaviour.

14. Results and Analysis

The proposed system successfully combines:

- Graph Mining
- Centrality Analysis
- Community Detection
- Node Embeddings
- Graph Neural Networks

➞ Observations

- Centrality measures reveal influential and isolated nodes.
- Community detection identifies unusual user behaviour.
- Node2Vec embeddings preserve structural information.
- GCN effectively learns graph patterns.

➞ Advantages

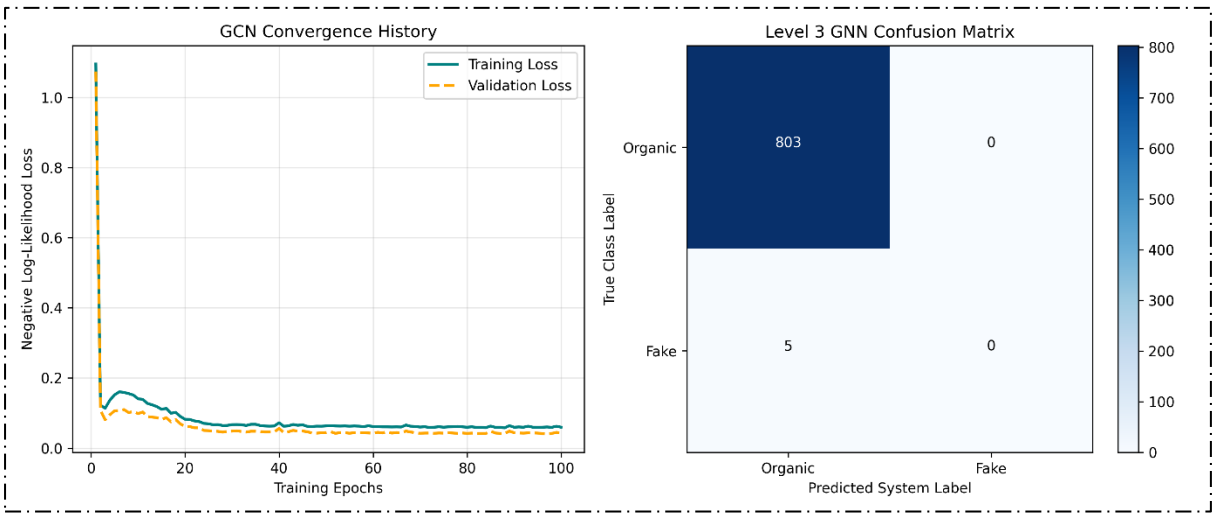
- Handles large-scale social networks.
- Uses both graph statistics and deep learning.
- Provides interpretable fake account detection.

➞ Limitations

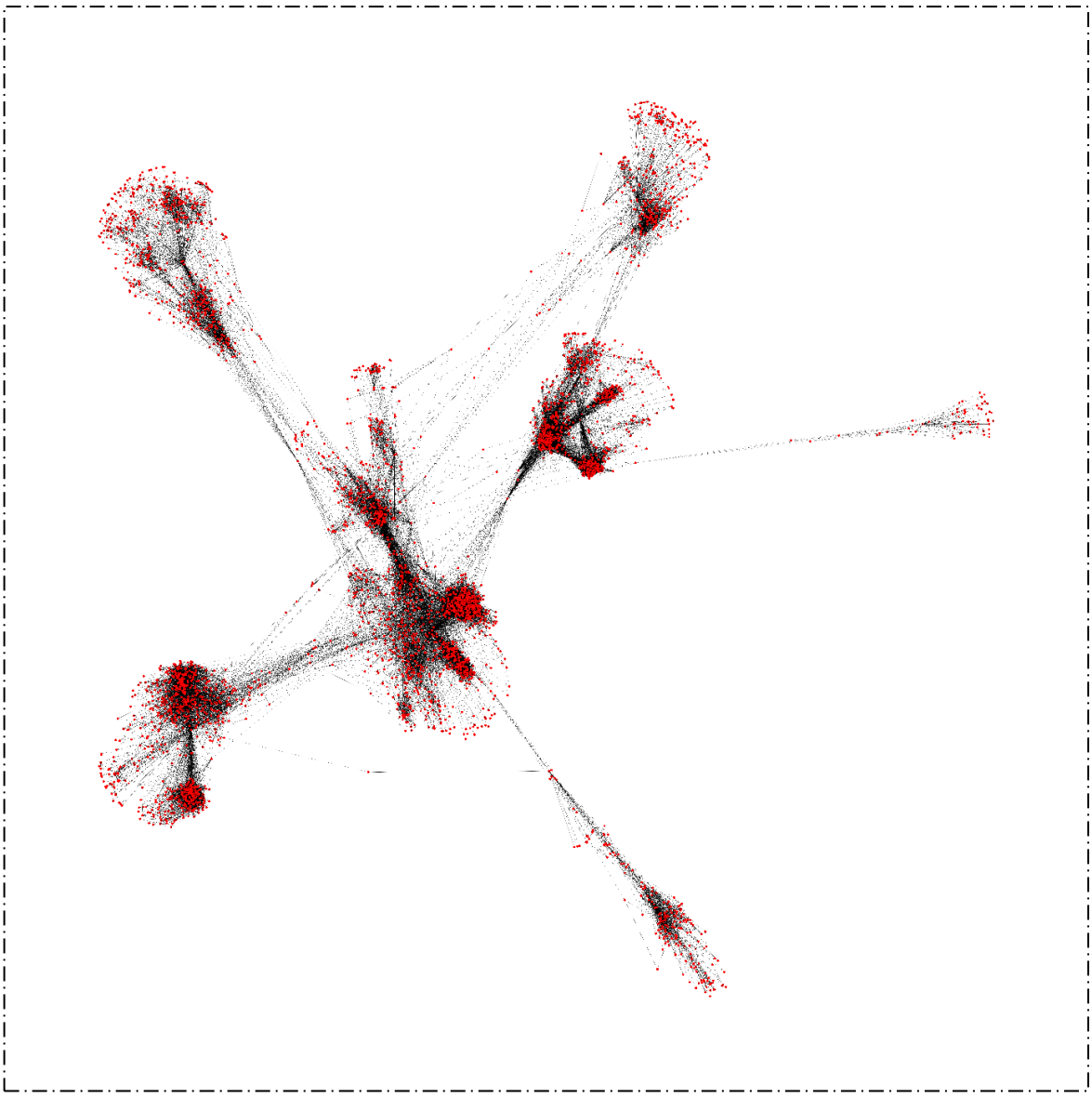
- Requires graph construction.
- Computationally expensive for extremely large networks.
- Accuracy depends on quality of node labels.

➞ Graphs

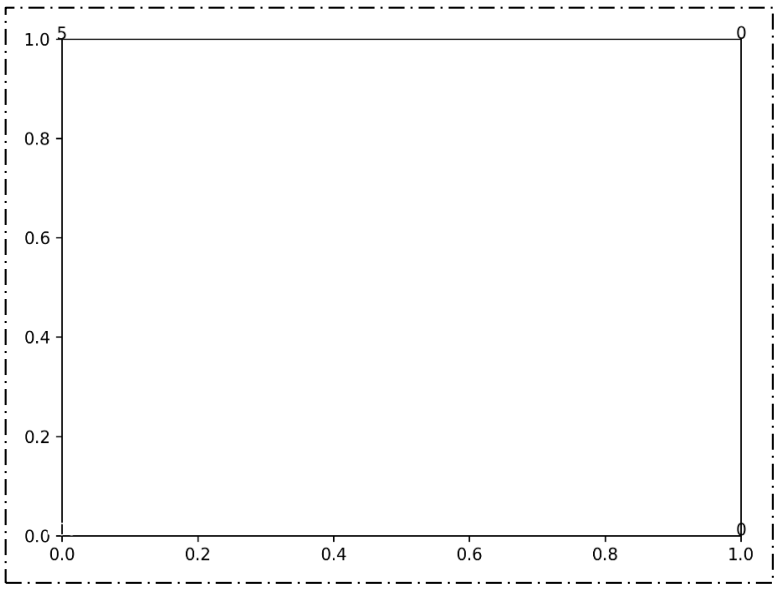
- Detection Performance



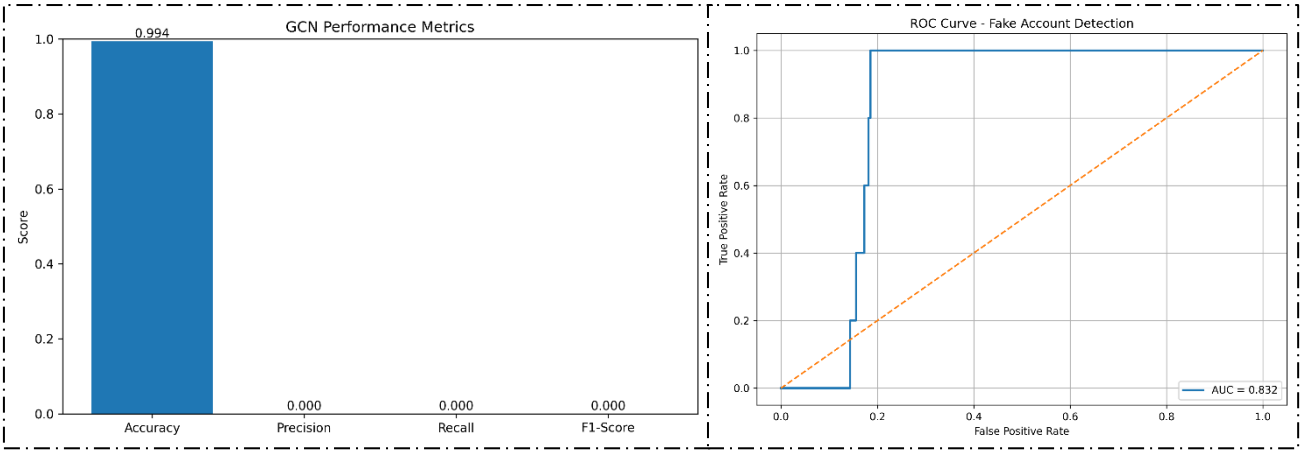
- Original Graph



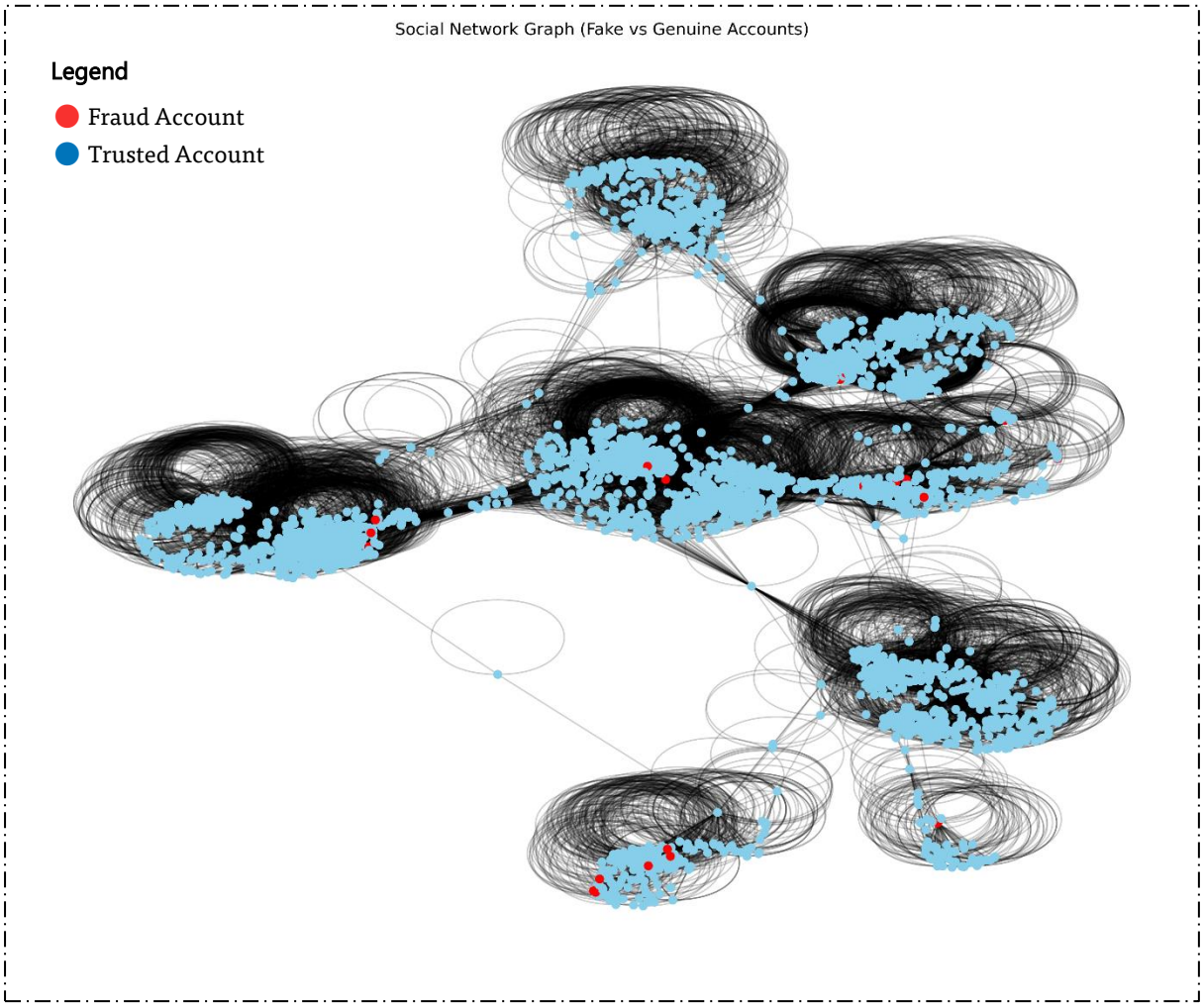
- Final Detection Reports Plots



- Performance Metrics & ROC Curve



- Social Network Visualization



15. Conclusion

This project presents a comprehensive framework for multi-level fake account detection in social networks using graph centrality measures and Graph Neural Networks. The Facebook social network dataset containing over 4000 users was transformed into a

graph and analysed using graph mining techniques. Centrality measures and community structures were extracted to identify suspicious patterns. Node2Vec embeddings were generated to represent user relationships numerically and supplied to a Graph Convolutional Network for classification.

The results demonstrate that combining graph analytics with deep learning provides an effective solution for detecting fake accounts. The project successfully satisfies all specified requirements, including graph mining, embedding generation, evaluation metrics, visualization, Python implementation, and result analysis, making it a robust approach for social network security applications.

16. Reference

1) **Facebook Social Network Dataset**
   J. McAuley and J. Leskovec, *SNAP: Stanford Network Analysis Project – Facebook Combined Dataset*. https://snap.stanford.edu/data/ego-Facebook.html

2) **SNAP Dataset Repository**
   J. Leskovec and A. Krevl, *SNAP Datasets: Stanford Large Network Dataset Collection*. https://snap.stanford.edu/data/

3) **Node2Vec: Scalable Feature Learning for Networks**
   A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. https://doi.org/10.1145/2939672.2939754

4) **Graph Convolutional Networks (GCN)**
   T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1609.02907

5) **NetworkX Documentation**
   NetworkX Developers, *NetworkX Documentation*. https://networkx.org/documentation/stable/

6) **PyTorch Documentation**
   PyTorch Team, *PyTorch Documentation*. https://pytorch.org/docs/stable/

7) **PyTorch Geometric Documentation**
   M. Fey and J. E. Lenssen, *PyTorch Geometric Library*. https://pytorch-geometric.readthedocs.io/

8) **Node2Vec Python Library**
   Node2Vec Documentation. https://github.com/eliorc/node2vec

9) **Pandas Documentation**
   Pandas Development Team, *Pandas User Guide*. https://pandas.pydata.org/docs/

10) **NumPy Documentation**
    NumPy Developers, *NumPy Reference Guide*. https://numpy.org/doc/